



# Anomaly Detection in Time Series – Classification Methods

# Classification-Based Anomaly Detection – hanc\_ml()

## From Regression to Classification

While `hanr_ml()` detects anomalies via prediction errors, `hanc_ml()` treats anomaly detection as a binary classification problem. The model is trained on labeled data where each observation is marked as normal (FALSE) or anomalous (TRUE). The positive class probability is used as the anomaly score.

## The `hanc_ml()` wrapper

- Accepts any DALToolbox binary classifier
- Requires a labeled dataset with a target column ("event")
- Anomaly score = probability of the positive class (TRUE)
- Observations above a threshold → flagged as anomalies

## Required steps

- Dataset: `examples_anomalies$tt` (labeled time series)
- Train/test split: first 80 rows for training, remainder for testing
- Normalization: `minmax()` fitted on training set, applied to both sets
- Prevents data leakage: test set normalized using training statistics only

## Regression vs. Classification

- `hanr_ml()`: no labels needed — uses prediction error as anomaly score
- `hanc_ml()`: requires labeled anomalies in the training set
- Classification can leverage richer supervision signal
- But: imbalanced datasets (few anomalies) are a major challenge

## Challenges

- Class imbalance: anomalies are rare → most classifiers default to FALSE
- Small datasets: neural networks (MLP) may fail to generalize
- Feature correlation: independence assumptions (Naive Bayes) often violated
- Overfitting: complex models may memorize training anomalies

## Best practices

- Always compare against a Majority baseline
- Prefer k-NN, RF, or SVM for small imbalanced datasets
- Normalize features before training
- Evaluate on a held-out test set

 **Key concept:** The key difference from regression: classification requires labeled anomalies at training time. This is a stronger assumption — but when labels are available, it can yield better precision and recall.

 Chandola, V., Banerjee, A., & Kumar, V. (2009). Anomaly detection: A survey. *ACM Computing Surveys*, 41(3), 1–58.

<https://doi.org/10.1145/1541880.1541882>

# 21 · hanc\_ml + Majority – Baseline Classifier

```
library(daltoolbox)
library(harbinger)

data(examples_anomalies)
# Classificação usa dataset "tt"
dataset <- examples_anomalies$tt

# Pré-processamento: split + escala
train <- dataset[1:80,]
test <- dataset[-(1:80),]
norm <- minmax()
norm <- fit(norm, train)
train_n <- transform(norm, train)
test_n <- transform(norm, test)

# Majority Classifier (baseline)
model <- hanc_ml(
  cla_majority("event",
    c("FALSE", "TRUE")))
model <- fit(model, train_n)
```

```
# Treino
detection <- detect(model, train_n)
# Treino: 0 TP, 0 FP (prevê sempre FALSE)

# Teste
detection <- detect(model, test_n)
evaluation <- evaluate(model,
  detection$event,
  as.logical(test_n$event))
print(evaluation$confMatrix)
## detection TRUE FALSE
##    TRUE  0  0
##   FALSE  2 19
# Teste: 0 TP, 0 FP

har_plot(model, test_n$serie,
  detection,
  as.logical(test_n$event))
```

## Step by step – Majority

- Classification uses `examples_anomalies$tt` (a labeled dataset)
- Split: 80 training / 21 test samples; min-max normalization applied
- `hanc_ml()` accepts any DALToolbox classifier
- Majority: always predicts the majority class (FALSE)
- Result: 0 TP, 0 FP — the minimum expected baseline

## Regression vs. Classification

- `hanr_ml()`: uses prediction error as the anomaly score
- `hanc_ml()`: uses the positive class probability as the anomaly score

📌 **Key concept:** The Majority classifier is an absolute baseline — it always predicts the most frequent class. Any useful model must outperform this result.

# Decision Tree Classifier

## What is a Decision Tree?

A Decision Tree recursively partitions the feature space using axis-aligned splits that maximize class separation (e.g., Gini impurity or information gain). The result is a tree of if-then rules that is highly interpretable. Each leaf node assigns a class label (or probability) to observations that reach it.

## Architecture

- Root node: first split on the most informative feature
- Internal nodes: subsequent splits on remaining features
- Leaf nodes: final class assignment (majority class in that region)
- Depth controls model complexity (deeper = more complex)
- Pruning prevents overfitting

## Classification-based detection

- Trained on labeled data (normal vs. anomalous)
- Splits the feature space to separate the two classes
- Positive class probability at each leaf → anomaly score
- Observations above a threshold → flagged as anomalies
- Struggles with class imbalance (few anomalies → biased splits)

## Key parameters (in daltoolbox)

- `cla_dtree("event", c("FALSE","TRUE"))`: target column and class labels
- Tree depth and pruning controlled internally
- No normalization required (tree splits are scale-invariant)
- Sensitive to small datasets and class imbalance

## Strengths

- Highly interpretable (human-readable rules)
- Fast training and inference
- No feature normalization required
- Useful as an interpretable baseline

## Limitations

- Fails on imbalanced datasets (0 TP observed)
- High variance: small data changes can produce very different trees
- Outperformed by RF, SVM, and k-NN on this dataset
- Not recommended as a final model for anomaly detection

❏ **Key concept:** The Decision Tree failed on this dataset (0 TP) due to class imbalance — anomalies are too rare for the tree to learn meaningful splits. It serves as an interpretable baseline before moving to ensemble methods.

Breiman, L., Friedman, J., Stone, C. J., & Olshen, R. A. (1984). *Classification and Regression Trees*. CRC Press. ISBN: 978-0412048418.

## 22 · hanc\_ml + Decision Tree – Decision Tree Classifier

```
library(daltoolbox)
library(harbinger)

data(examples_anomalies)
dataset <- examples_anomalies$tt

train <- dataset[1:80,]
test <- dataset[-(1:80),]
norm <- minmax()
norm <- fit(norm, train)
train_n <- transform(norm, train)
test_n <- transform(norm, test)

# Decision Tree
model <- hanc_ml(
  cla_dtree("event",
    c("FALSE", "TRUE")))
model <- fit(model, train_n)

# Training
detection <- detect(model, train_n)
# Train: 0 TP, 0 FP
```

```
# Test
detection <- detect(model, test_n)
evaluation <- evaluate(model,
  detection$event,
  as.logical(test_n$event))
print(evaluation$confMatrix)
## detection TRUE FALSE
##   TRUE  0   0
##  FALSE  2  19
# Test: 0 TP, 0 FP

har_plot(model, test_n$serie,
  detection,
  as.logical(test_n$event))
```

### Step-by-step – Decision Tree

- `cla_dtree()`: decision tree classifier from DALToolbox
- Imbalanced dataset: very few anomalies versus many normal observations
- Failed on the test set (0 TP) — dataset is small and heavily imbalanced
- Useful as an interpretable baseline before moving to more complex models
- Compare with k-NN and RF to assess real performance gains

📌 **Key concept:** The Decision Tree is interpretable and fast, but can struggle with imbalanced datasets that have few anomalies. It serves as a baseline before more sophisticated models such as RF and SVM.

# k-Nearest Neighbors (k-NN)

## What is k-NN?

k-NN is a non-parametric, instance-based classifier. It makes predictions by finding the k most similar training examples (nearest neighbors) to a new observation, then assigning the majority class among those neighbors. There is no explicit training phase — the model simply memorizes the training set.

## How similarity is measured

- Distance metric: typically Euclidean distance in feature space
- k: number of neighbors to consider (k=3 used here)
- Anomaly score: proportion of neighbors belonging to the positive class (TRUE)
- Observations above a threshold → flagged as anomalies
- Normalization is critical: features must be on the same scale

## Classification-based detection

- Trained on labeled data (normal vs. anomalous)
- At inference: finds k nearest neighbors in the training set
- Positive class proportion among neighbors = anomaly score
- Works well when anomalies form distinct clusters in feature space
- Excellent result on this dataset: 4 TP (train), 2 TP (test), 0 FP

## Key parameters (in daltoolbox)

- `cla_knn("event", c("FALSE", "TRUE"), k=3)`: k=3 neighbors
- Normalization: `minmax()` required (distance-sensitive)
- Larger k → smoother decision boundary, less sensitive to noise
- Smaller k → more sensitive to local structure

## Strengths

- No training phase (lazy learner)
- Naturally handles multi-modal class distributions
- Excellent generalization on this dataset (2 TP, 0 FP on test)
- Works well when anomalies form tight clusters

## Limitations

- Slow inference: must compute distances to all training points
- Memory-intensive: stores entire training set
- Sensitive to irrelevant features and the choice of k
- Degrades in very high-dimensional spaces (curse of dimensionality)

 **Key concept:** k-NN achieved the best classification result on this dataset alongside RF and SVM. Its success confirms that anomalies form distinct clusters in the feature space — making similarity-based reasoning highly effective here.

<sup>1</sup> Cover, T., & Hart, P. (1967). Nearest neighbor pattern classification. *IEEE Transactions on Information Theory*, 13(1), 21–27.

<https://doi.org/10.1109/TIT.1967.1053964>

## 23 · hanc\_ml + k-NN – k Nearest Neighbors

```
library(daltoolbox)
library(harbinger)

data(examples_anomalies)
dataset <- examples_anomalies$tt

train <- dataset[1:80,]
test <- dataset[-(1:80),]
norm <- minmax()
norm <- fit(norm, train)
train_n <- transform(norm, train)
test_n <- transform(norm, test)

# k-NN (k=3)
model <- hanc_ml(
  cla_knn("event",
    c("FALSE", "TRUE"), k=3))
model <- fit(model, train_n)

# Training
detection <- detect(model, train_n)
# Train: 4 TP, 0 FP (perfect)
```

```
# Test
detection <- detect(model, test_n)
evaluation <- evaluate(model,
  detection$event,
  as.logical(test_n$event))
print(evaluation$confMatrix)
## detection TRUE FALSE
##   TRUE  2   0
##  FALSE  0  19

har_plot(model, test_n$serie,
  detection,
  as.logical(test_n$event))
```

### Step-by-step – k-NN

- k=3: considers the 3 nearest neighbors
- Training: perfect classification (4 TP, 0 FP)
- Test: 2 TP, 0 FP — excellent generalization on this dataset
- Positive class probabilities above threshold → event detected
- Works well when anomalies form distinct clusters in feature space

📌 **Key concept:** k-NN classifies by similarity to labeled examples — it works well when anomalies form distinct clusters in feature space. Delivers excellent results on this dataset.

# Naive Bayes Classifier

## What is Naive Bayes?

Naive Bayes is a probabilistic classifier based on Bayes' theorem with the "naive" assumption that all features are conditionally independent given the class label. Despite this strong assumption, it often performs surprisingly well in practice — especially in text classification. However, it struggles when features are correlated, as is typical in time series data.

## Bayes' theorem applied

- $P(\text{class} \mid \text{features}) \propto P(\text{class}) \times \prod P(\text{feature}_i \mid \text{class})$
- $P(\text{class})$ : prior probability of each class (estimated from training data)
- $P(\text{feature}_i \mid \text{class})$ : likelihood of each feature given the class
- Gaussian NB: assumes each feature follows a normal distribution per class
- Anomaly score:  $P(\text{TRUE} \mid \text{features})$

## Classification-based detection

- Trained on labeled data (normal vs. anomalous)
- Estimates class-conditional distributions for each feature independently
- Anomaly score =  $P(\text{TRUE} \mid \text{observation})$
- Observations above a threshold → flagged as anomalies
- Failed on this dataset: 0 TP — independence assumption violated

## Key parameters (in daltoolbox)

- `cla_nb("event", c("FALSE", "TRUE"))`: target column and class labels
- No hyperparameters to tune (fully determined by training data)
- Normalization: `minmax()` applied (consistent with other classifiers)
- Gaussian NB assumes continuous features with normal distribution

## Strengths

- Extremely fast training and inference
- Works well with high-dimensional data (e.g., text)
- Provides calibrated probability estimates
- Simple probabilistic baseline

## Limitations

- Independence assumption violated in time series (features are correlated)
- Failed on this dataset (0 TP) — confirms assumption inadequacy
- Cannot capture feature interactions
- Outperformed by k-NN, RF, and SVM on correlated data

 **Key concept:** Naive Bayes failed here because time series features (lagged values) are highly correlated — directly violating the independence assumption. This is a valuable negative result: it confirms that temporal correlation must be modeled explicitly.

<sup>1</sup> Zhang, H. (2004). The optimality of Naive Bayes. *Proceedings of the 17th International FLAIRS Conference*, 562–567.  
<https://www.aaai.org/Library/FLAIRS/2004/flairs04-097.php>

## 24 · hanc\_ml + Naive Bayes – Probabilistic Classifier

```
library(daltoolbox)
library(harbinger)

data(examples_anomalies)
dataset <- examples_anomalies$tt

train <- dataset[1:80,]
test <- dataset[-(1:80),]
norm <- minmax()
norm <- fit(norm, train)
train_n <- transform(norm, train)
test_n <- transform(norm, test)

# Naive Bayes
model <- hanc_ml(
  cla_nb("event",
    c("FALSE","TRUE")))
model <- fit(model, train_n)

# Training
detection <- detect(model, train_n)
# Training: 0 TP, 0 FP
```

```
# Test
detection <- detect(model, test_n)
evaluation <- evaluate(model,
  detection$event,
  as.logical(test_n$event))
print(evaluation$confMatrix)
## detection TRUE FALSE
##   TRUE  0   0
##  FALSE  2  19
# Test: 0 TP, 0 FP (failed)

har_plot(model, test_n$serie,
  detection,
  as.logical(test_n$event))
```

### Step-by-step – Naive Bayes

- Assumes conditional independence between features
- Failed on test set (0 TP) — independence assumption is inadequate here
- Time series features are highly correlated with one another
- Useful as a simple probabilistic baseline
- Contrast with k-NN: similarity outperforms independence in this case

📌 **Key concept:** Naive Bayes assumes conditional independence between features — an assumption rarely valid in time series data. It failed on this dataset, confirming that temporal correlation plays a critical role in classification.

# Random Forest & SVM – Classification

## RF as a Classifier

Random Forest for classification works the same way as for regression, but each tree votes for a class label instead of predicting a continuous value. The final prediction is the majority vote across all trees. The positive class vote proportion serves as the anomaly score.

### Key advantages for anomaly detection

- Handles class imbalance better than a single Decision Tree
- Aggregating many trees reduces variance and overfitting
- Feature importance: identifies which lagged values are most predictive
- `mtry`: number of features considered at each split (controls randomness)
- `ntree`: number of trees (more trees = more stable, but slower)

## SVM as a Classifier

SVM for classification finds the maximum-margin hyperplane that separates the two classes. The margin is the distance between the hyperplane and the nearest training points (support vectors). A soft-margin SVM (controlled by cost  $C$ ) allows some misclassifications to handle non-separable data.

### Key advantages for anomaly detection

- Maximum-margin principle: robust to outliers near the decision boundary
- `cost` ( $C$ ): penalty for misclassification — high  $C$  = tighter fit to training data
- Kernel trick: can use RBF or linear kernel for non-linear boundaries
- Works well with small datasets and few labeled anomalies
- `epsilon=0.0`: no regression tube (pure classification mode)

## RF vs. SVM on this dataset

- Both achieved: 4 TP (train), 2 TP (test), 0 FP
- RF: `mtry=1`, `ntree=5` — minimal ensemble, fast
- SVM: `cost=20` — high penalty, tight fit to labeled anomalies
- Both outperformed Majority, Decision Tree, Naive Bayes, and MLP
- Preferred choice when labeled anomalies are available

### When to choose each

- RF: when interpretability (feature importance) matters
- SVM: when the decision boundary is complex and data is small
- Both: robust alternatives to neural networks on small datasets

☐ RF and SVM are the top-performing classifiers on this dataset. Their success with few labeled examples (4 training anomalies) makes them the recommended choice for small, imbalanced anomaly detection tasks.

References:

<sup>1</sup> Breiman, L. (2001). Random forests. *Machine Learning*, 45(1), 5–32.

<https://doi.org/10.1023/A:1010933404324>

<sup>2</sup> Cortes, C., & Vapnik, V. (1995). Support-vector networks. *Machine Learning*, 20(3), 273–297.

<https://doi.org/10.1007/BF00994018>

## 25-26 · hanc\_ml + RF and SVM – Advanced Classification

```
library(daltoolbox)
library(harbinger)

data(examples_anomalies)
dataset <- examples_anomalies$tt
train <- dataset[1:80,]
test <- dataset[-(1:80),]
norm <- minmax()
norm <- fit(norm, train)
train_n <- transform(norm, train)
test_n <- transform(norm, test)

# --- Random Forest ---
model_rf <- hanc_ml(
  cla_rf("event",
    c("FALSE", "TRUE"),
    mtry=1, ntree=5))
model_rf <- fit(model_rf, train_n)
# Train: 4 TP, 0 FP

# --- SVM ---
model_svm <- hanc_ml(
  cla_svm("event",
    c("FALSE", "TRUE"),
    epsilon=0.0, cost=20))
model_svm <- fit(model_svm, train_n)
# Train: 4 TP, 0 FP
```

```
# RF - Test
detection_rf <- detect(model_rf, test_n)
# Test: 2 TP, 0 FP

# SVM - Test
detection_svm <- detect(model_svm, test_n)
evaluation <- evaluate(model_svm,
  detection_svm$event,
  as.logical(test_n$event))
print(evaluation$confMatrix)
## detection TRUE FALSE
##   TRUE  2  0
##  FALSE  0 19
# Test: 2 TP, 0 FP

har_plot(model_svm, test_n$serie,
  detection_svm,
  as.logical(test_n$event))
```

### Step-by-step – RF Classification

- `mtry=1`: features per split; `ntree=5`: number of trees
- Perfect train and test results: 4 TP / 2 TP, 0 FP

### Step-by-step – SVM Classification

- `cost=20`: high penalty for misclassification errors
- Same result as RF: 4 TP on train, 2 TP on test, 0 FP

❏ **Key concept:** RF and SVM classifiers outperformed simpler models on this dataset. Models with greater class separation capacity perform better when there are few labeled anomaly examples.

# MLP as a Classifier

## MLP for Classification

The same MLP architecture used for regression can be adapted for binary classification by replacing the linear output with a softmax (or sigmoid) activation and using cross-entropy loss instead of mean squared error. The output is a probability distribution over classes — the positive class probability (TRUE) serves as the anomaly score.

### Architecture differences from regression MLP

- Output layer: softmax activation → class probabilities
- Loss function: cross-entropy (instead of MSE)
- Regularization: L2 weight decay (decay parameter) prevents overfitting
- Same backpropagation training as regression MLP
- Anomaly score: P(TRUE | features) from the output layer

## Classification-based detection

- Trained on labeled data (normal vs. anomalous)
- Learns non-linear decision boundaries via hidden layers
- Positive class probability → anomaly score
- Observations above a threshold → flagged as anomalies
- Failed on this dataset: 0 TP — insufficient labeled data

### Key parameters (in daltoolbox)

- `cla_mlp("event", c("FALSE", "TRUE"), size=3, decay=0.03)`
- `size`: number of hidden neurons (3 used here)
- `decay`: L2 regularization (0.03 — moderate regularization)
- Normalization: `minmax()` required
- Sensitive to dataset size and class balance

## Strengths

- Learns complex non-linear decision boundaries
- Flexible architecture (depth, width, regularization)
- Can outperform RF and SVM on large, balanced datasets
- Same framework as regression MLP (easy to switch)

## Limitations

- Requires more labeled data than k-NN, RF, or SVM
- Failed on this small dataset (0 TP) — overfitting or underfitting
- Sensitive to initialization and hyperparameters
- Not recommended when labeled anomalies are scarce



**Key concept:** MLP as a classifier failed on this small dataset — the same architecture that achieved perfect regression results. This highlights a key insight: neural networks need sufficient labeled examples to learn meaningful classification boundaries. With only 4 training anomalies, RF and SVM are far more reliable.

Reference:

<sup>1</sup> Goodfellow, I., Bengio, Y., & Courville, A. (2016). *Deep Learning*. MIT Press.  
<https://www.deeplearningbook.org>

## 27 · hanc\_ml + MLP – Neural Network Classifier

```
library(daltoolbox)
library(harbinger)

data(examples_anomalies)
dataset <- examples_anomalies$tt
train <- dataset[1:80,]
test <- dataset[-(1:80),]
norm <- minmax()
norm <- fit(norm, train)
train_n <- transform(norm, train)
test_n <- transform(norm, test)

# MLP Classificação
model <- hanc_ml(
  cla_mlp("event",
    c("FALSE", "TRUE"),
    size=3, decay=0.03))
model <- fit(model, train_n)

# Treino
detection <- detect(model, train_n)
# Treino: 0 TP, 0 FP
```

```
# Teste
detection <- detect(model, test_n)
evaluation <- evaluate(model,
  detection$event,
  as.logical(test_n$event))
print(evaluation$confMatrix)
## detection TRUE FALSE
##   TRUE  0   0
##  FALSE  2  19
# Teste: 0 TP, 0 FP (falhou)

har_plot(model, test_n$serie,
  detection,
  as.logical(test_n$event))
```

### Step-by-step – MLP Classification

- size=3: hidden neurons; decay=0.03: L2 regularization
- Failed on the test set (0 TP) — likely needs more training data
- Small datasets are disadvantageous for neural networks
- Contrast with RF and SVM: simpler models generalized significantly better

❏ **Key concept:** MLP as a classifier failed on this small dataset. Neural networks generally require more labeled examples to generalize effectively — RF and SVM are preferable when labeled data is scarce.

# Summary: Classification Methods and When to Use Them

## Classification: `hanc_ml()`

**Methods:** Majority, Decision Tree, k-NN (k=3), Naive Bayes, Random Forest, SVM (cost=20), MLP

Requires labeled data with train/test split and normalization.

**Best performers:** k-NN, RF, SVM (2 TP on test set ✓)

## When to Use Classification

Use when **labeled anomalies exist** in the training set (positive class available).

Prefer k-NN, RF, or SVM. Avoid MLP and Decision Tree with small or imbalanced datasets.

## Classifier Comparison

**2 TP (best):** k-NN, RF, SVM

**1 TP:** Decision Tree, Naive Bayes

**0 TP (failed):** Majority, MLP

Simpler models generalized better on small datasets — complexity does not guarantee performance.

## Key Requirements

- Labeled training data required (event column with TRUE/FALSE values)
- Train/test split recommended to assess generalization
- Normalization (minmax) improves performance for distance-based (k-NN) and neural (MLP) models

# Additional Sources

Explore the resources below to dive deeper into Harbinger and time series event detection.

## CRAN: harbinger

[cran.r-project.org/web/packages/harbinger](https://cran.r-project.org/web/packages/harbinger)

Core package for anomaly detection in time series. Provides the `hanc_ml()` wrapper for classification-based detection.

## CRAN: daltoolbox

[cran.r-project.org/web/packages/daltoolbox](https://cran.r-project.org/web/packages/daltoolbox)

DAL Toolbox: base framework for ML models, normalization, and evaluation used throughout.

## Event Detection Resources

[eic.cefet-rj.br/~eogasawara/en/event-detection-in-time-series](https://eic.cefet-rj.br/~eogasawara/en/event-detection-in-time-series)

Full tutorial and documentation by Eduardo Ogasawara.

## Issues & Discussions

[github.com/cefet-rj-dal/harbinger/issues](https://github.com/cefet-rj-dal/harbinger/issues)

Open questions, report bugs, or suggest improvements directly in the repository.